JavaProgramming ကို ၁၉၉၅ ခုနှစ်တွင် Sun MicroSystem ကစတင်ခဲ့တယ်။Java Enterprise Edition ကိုဖန်တီးရန် Java Architect တစ်ချို့က ကြိုးစားအားထုတ်နေပြီး Container Component Architecture ကိုအသုံးပြုရန်ရွေးချယ်ကြတယ်။Container မှ Enterprise Level Services များဖြစ်တဲ့ Pooling ,Thread Safety, State Management, Transactions, Security, Interceptors ,Caching And Performance အစရှိသည်များကို ထောက်ပံ့ရန်ရေးသားထားသည်။ ပထမဆုံး J2EE 1.3 SDK ကို 2001 April တွင်ဖြန့်ချိနိုင်တယ်။ ရေးသားရန်မလွယ်ကူခြင်း ၊ interfaces အမြောက်အများကို implements လုပ်ရန်လိုအပ်နေခြင်း ၊ Framework ကို အမှီပြုနေသော Class များကို extends လုပ်ရန်လိုအပ်ခြင်းတို့က Java Enterprise Edition ကိုနာမည်ဆိုးဖြင့်သာ ကျော်ကြားစေတယ်။ဆက်လက်ကြိုးစားကြပြီး 2002 December တွင် J2EE 1.4 သို့အဆင့်မြှင့်တင်နိုင်ခဲ့သော်လည်း မကြာခနဖြစ်နေတဲ့ deployment error သည်ပြဿနာတစ်ခုဖြစ်လာသည်။

2002 October တွင် Rod Johnson က Expert One-on-One J2EE Design and Development စာအုပ်ကိုရေးသားပြီး J2EE ၏ချို့ယွင်းချက်များကို ထောက်ပြတယ်။ သူ့စာအုပ်တွင် ရိုးစင်းလှသည့် POJO( plain old java objects) များနှင့် interface များကိုသာ အခြေခံပြီး ပိုမိုကောင်းမွန်သည့် architecture ကိုသာ မျော်လင့်မိကြောင်းရေးသားထားသည်။စာအုပ်တွင် BeanFactory ၊ ApplicationContext ကဲ့သို့အရေးပါသည့် interfaces များ classes များကြောင်းရေ ၃၀၀၀၀ ကျော်ပါဝင်သည်။Juergen Hoeller နှင့် Yann Caroff တို့ကို Rod ကစည်းရုံးပြီး သူရေးသားထားတဲ့ code များကို အခြေခံကာ ၂၀၀၃ ဖေဖော်ဝါရီတွင် OpenSource Project အဖြစ်စတင်တယ်။June 2003 တွင် Spring 0.9 ကို Apaceh 2.0 လိုင်စင်ဖြင့်လည်းကောင်း March 2004 တွင် Spring 1.0 ကိုဖြန့်ချိနိုင်ခဲ့သည်။ Rod Johnson , Juergen Hoeller, Keith Donald and Colin Sampaleanu တို့ပေါင်းပြီး ၂၀၀၄ တွင် interface21 ကိုတည်ထောင်ပြီး Spring Framework သည် မွေးဖွားလာပြီဖြစ်သည်။

Spring Framework သည် Java EE ကဲ့သို့ပင် Container Component Architecture ကိုပင်အခြေခံထားသော်လည်း ကွားခြားချက်မှာ Java EE သည် Application Server like GlassFish, WildFly , Weblogic တို့မရှိမဖြစ်လိုအပ်နေခြင်းဖြစ်ပြီး လည်းကောင်း Servers များကိုယ်တိုင်ပင် Container အဖြစ်အလုပ်လုပ်ဆောင်သည်။ BeanFactory interface သည် Spring Bean Container ၏အခြေခံအကျဆုံး interface ဖြစ်ပြီး Spring Container ဖြစ်သည်။ ApplicationContext interface သည် BeanFactory interface ၏ subinterface ဖြစ်သည်။ i18n ၊ application level lifecycle events Validation နှင့် Enterprise Level Service တစ်ချို့ကိုထောက်ပံ့ပေးသည်။ထို့ကြောင့် Spring ၏

Container သည် interface သာဖြစ်သည်လို့ ကောက်ချက်ချမှတ်ယူနိုင်ပေသည်။

SpringBeanFactory တွင် အသုံးအများဆုံးပုံစံအဖြစ် Xml Bean Factory ကို အသုံးပြုကြသည်။ Xml BeanFactory ကိုရေးသားရန် Ide တွင်း SpringBeanConfigurationFile ကိုအသုံးပြုရမည်ဖြစ်သည်။ SpringBeanContainer အတွင်းတွင် Java Class များသည် Spring Bean အဖြစ် တည်ရှိမည်။ Spring Container က စီမံနေတဲ့ Spring Bean Component အဖြစ်တည်ရှိနေမည်။ Spring Bean Factory က Java Class များကို Spring Bean အဖြစ် ပြောင်းလဲပေးလိုက်သည်။နမူနာ အဖြစ်

```xml
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://www.springframework.org/schema/beans http://www.springframework.org/schema/beans/spring-beans.


    <bean id="student" class="demo.Student"></bean>

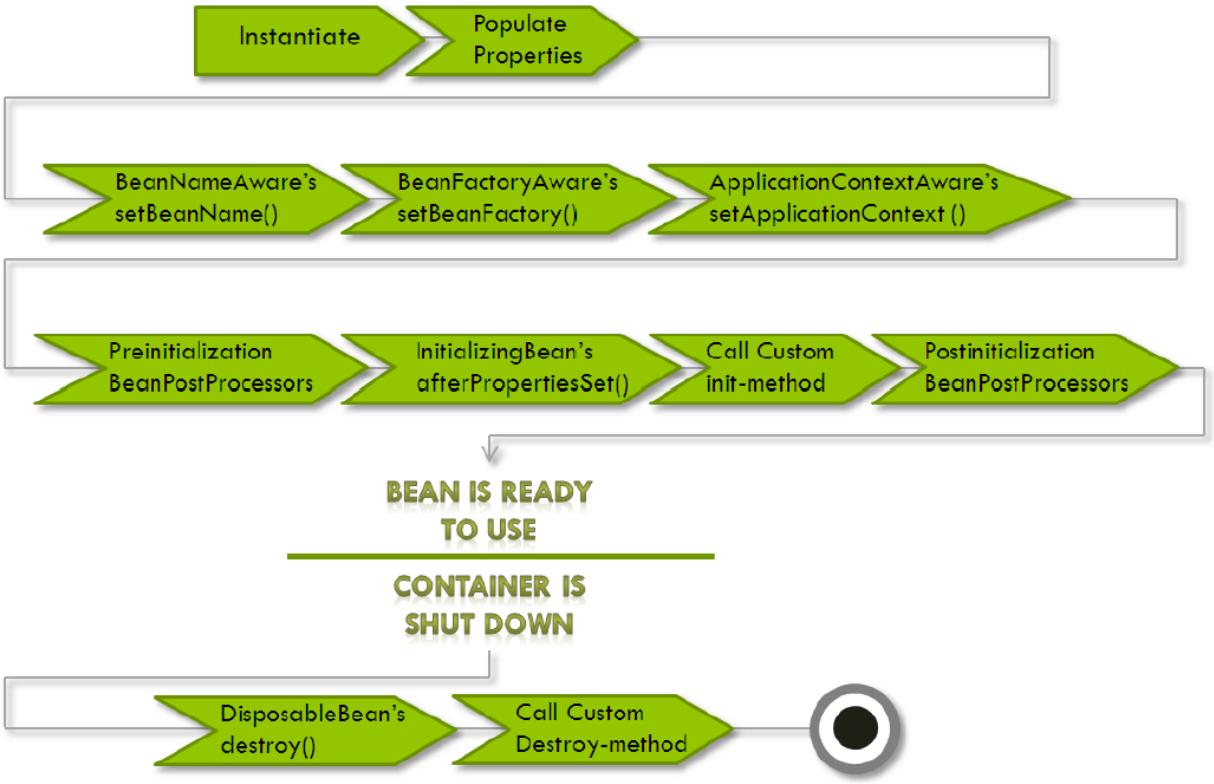

    <bean id="myStudentService" class="demo.MyStudentService"></bean>


    <bean id="studentService" class="demo.StudentService"
        init-method="customInit" destroy-method="customDestory">
        <property name="student" ref="student"></property>
    </bean>


</beans>

```

Student , MyStudentService နှင့် StudentService တို့သည် Spring Bean Container ကဖန်တီးလိုက်သော Spring Bean များဖြစ်သွားပြီဖြစ်သည်။သာမန်ရိုးရိုး Java classes များမဟုတ်တော့ပေ။ Spring Bean Factory က Java Classes များကို Spring Bean အဖြစ်ဖန်တီးပုံအဆင့်ဆင့်ကို လေ့လာကြပါစို့။




SpringBeanFactory သည် ပထမဆုံး Java Class ၏ Default Constructor ကိုအသုံးပြုပြီး Object တည်ဆောက်သည်။ instantiate အဆင့်ပင်ဖြစ်သည်။

Spring Container သည် bean ၏ properties များကို injects လုပ်ခြင်းများကို Porpulate Properties အဆင့်တွင် အလုပ်လုပ်ဆောင်သည်။

နောက် Spring Container သည် Bean Name ကိုသတ်မှတ်ပြီး BeanNameAware interface ၏ setBeanName(String name) ကိုခေါ်ဆိုသည်။

BeanFactoryAware interface ၏ setBeanFactory(BeanFactory beanFactory) ကိုခေါ်ဆိုပြီး bean instance ကို
BeanFactory နှင့် ချိတ်ဆက်ရန်ထောက်ပံ့ပေးသည်။

ApplicationContextAware ၏ setApplicationContext(ApplicationContext applicationContext)ကိုခေါ်ဆိုပြီး bean အလုပ်လုပ်ဆောင်မည့် ApplicationContext ကို သတ်မှတ်သည်။ပုံမှန်အားဖြင့် BeanFactory က Bean ကို object တည်ဆောက်တိုင်းခေါ်ဆို မှာဖြစ်သည်။

နောက် BeanPostProcessor interface ၏ postProcessBeforeInitialization( ) ကိုခေါ်ဆိုသည်။

```java
package demo;

import org.springframework.beans.BeansException;
import org.springframework.beans.factory.config.BeanPostProcessor;

public class MyStudentService implements BeanPostProcessor{

    @Override
    public Object postProcessAfterInitialization(Object bean, String beanName) throws BeansException {
        // TODO Auto-generated method stub

        System.out.println("Bean Name: postProcessAfterInitialization"+ beanName);
        return BeanPostProcessor.super.postProcessAfterInitialization(bean, beanName);
    }

    @Override
    public Object postProcessBeforeInitialization(Object bean, String beanName) throws BeansException {
        // TODO Auto-generated method stub
        System.out.println("Bean Name: postProcessBeforeInitialization"+ beanName);
        return BeanPostProcessor.super.postProcessBeforeInitialization(bean, beanName);
    }


}
```

မည်သည့် Bean ဖန်တီးတည်ဆောက်သည်မဆို ApplicationContext သည် အလိုအလျောက်သိရှိ မည်ဖြစ်ပြီး Spring Bean တိုင်းအတွက်အလုပ်လုပ်ဆောင်မည့် method ဖြစ်သည်။

ထို့နောက်တွင် ကို အထက်က method များကိုခေါ်ဆို ပြီးနောက် BeanFactory က InitializingBean ၏ afterPropertySet( ) ကိုခေါ်ဆိုလိုက်သည်။

```java
package demo;

import org.springframework.beans.factory.DisposableBean;

public class StudentService implements InitializingBean,DisposableBean{

    private Student student;

    public Student getStudent() {
        return student;
    }

    public void setStudent(Student student) {
        this.student = student;
    }

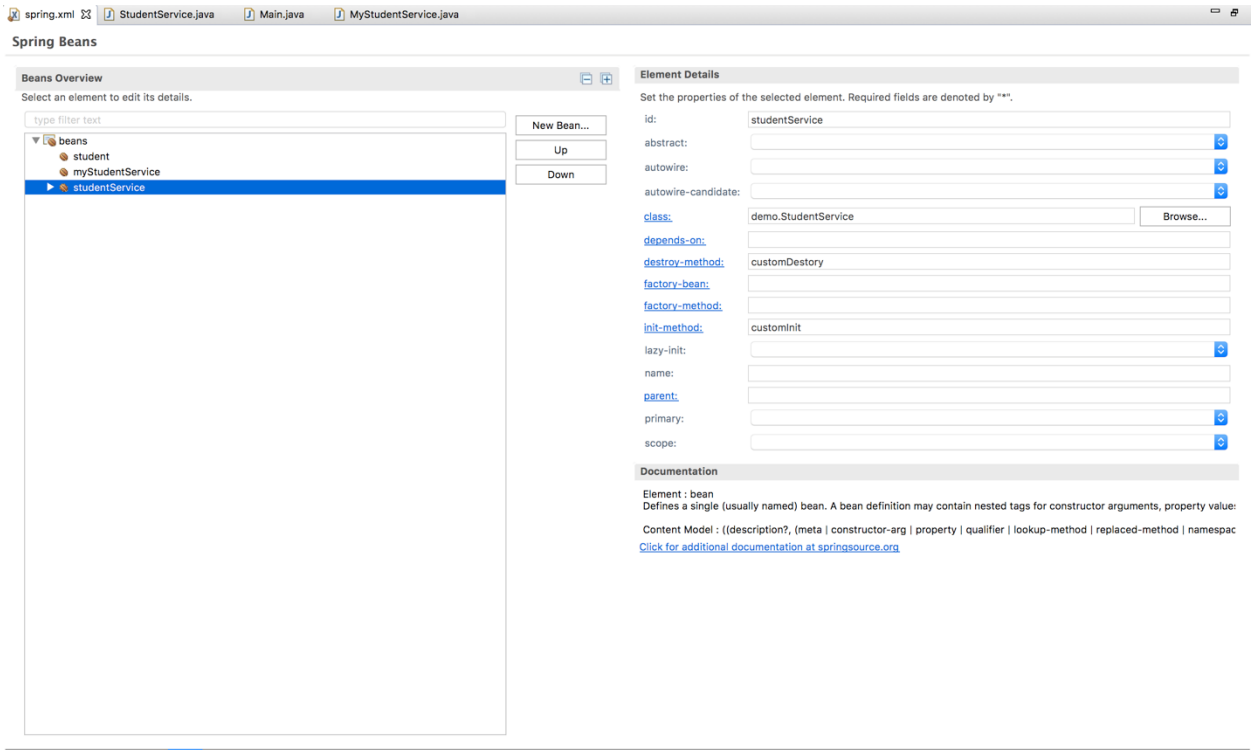
    @Override
    public void afterPropertiesSet() throws Exception {
        System.out.println("student service: afterPropertiesSet");
        if(student.getName()==null) {
            student.setName("Thaw Thaw");
        }
    }
```

BeanFactory သည် Custom init method ကိုခေါ်ဆိုလိုက်သည်။

```java
    public void customInit() {
        System.out.println("StudentService customInit");
    }

    public void customDestory() {
        System.out.println("StudentService customDestory");
    }

}
```




ထို့နောက်တွင် BeanPostProcessor interface ၏ postProcessAfterInitialization( ) ကိုခေါ်ဆိုသည်။

```
1  package demo;
2
3  import org.springframework.beans.BeansException;
4  import org.springframework.beans.factory.config.BeanPostProcessor;
5
6  public class MyStudentService implements BeanPostProcessor{
7
8      @Override
9      public Object postProcessAfterInitialization(Object bean, String beanName) throws BeansException {
10         // TODO Auto-generated method stub
11
12         System.out.println("Bean Name: postProcessAfterInitialization"+ beanName);
13         return BeanPostProcessor.super.postProcessAfterInitialization(bean, beanName);
14     }
15
16     @Override
17     public Object postProcessBeforeInitialization(Object bean, String beanName) throws BeansException {
18         // TODO Auto-generated method stub
19         System.out.println("Bean Name: postProcessBeforeInitialization"+ beanName);
20         return BeanPostProcessor.super.postProcessBeforeInitialization(bean, beanName);
21     }
22
23
24
25 }
26
```

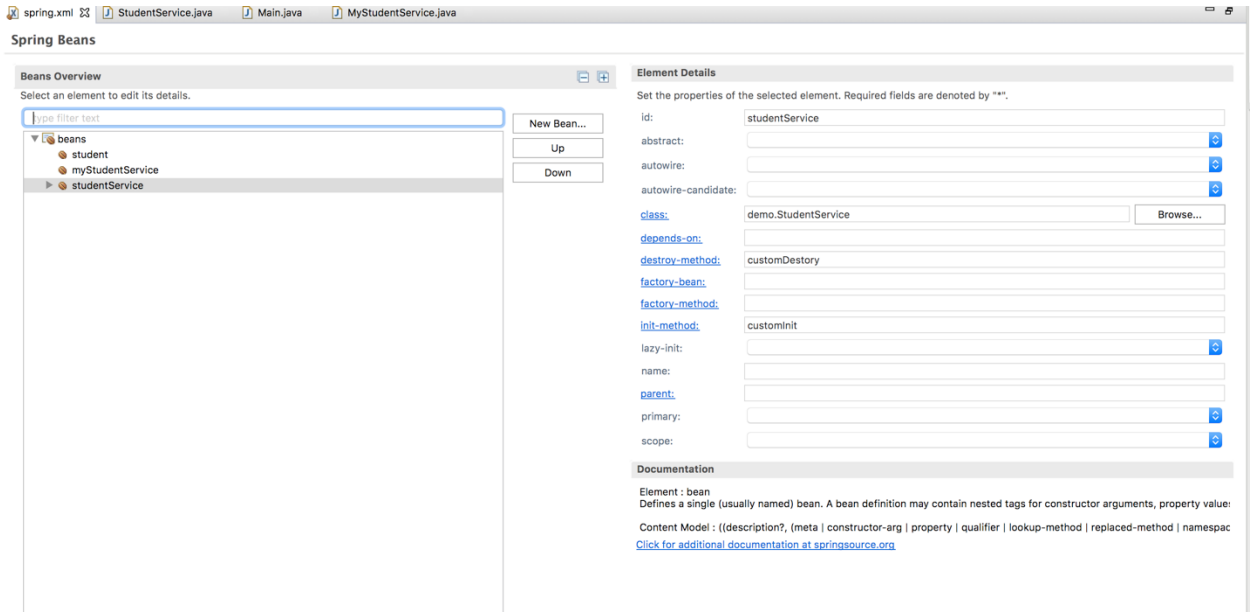
ယခုအဆင့်တွင် အဆင်သင့်အသုံးပြုနိုင်သည့် Spring Bean ကိုရရှိမည်ဖြစ်သည်။

Container shut down ပြုလုပ်သည့်အခါတွင် DisposableBeans interface ၏ destroy( ) ကိုခေါ်ဆိုသည်။

```
1  package demo;
2
3  import org.springframework.beans.factory.DisposableBean;
4  import org.springframework.beans.factory.InitializingBean;
5
6  public class StudentService implements InitializingBean,DisposableBean{
7
8      private Student student;
9
10     public Student getStudent() {
11         return student;
12     }
13
14     public void setStudent(Student student) {
15         this.student = student;
16     }
17
18     @Override
19     public void afterPropertiesSet() throws Exception {
20         System.out.println("student service: afterPropertiesSet");
21         if(student.getName()==null) {
22             student.setName("Thaw Thaw");
23         }
24     }
25
26     @Override
27     public void destroy() throws Exception {
28         System.out.println("Student Service Destory!");
29
30     }
31
32
33
34     public void customInit() {
35         System.out.println("StudentService customInit");
36     }
37
```

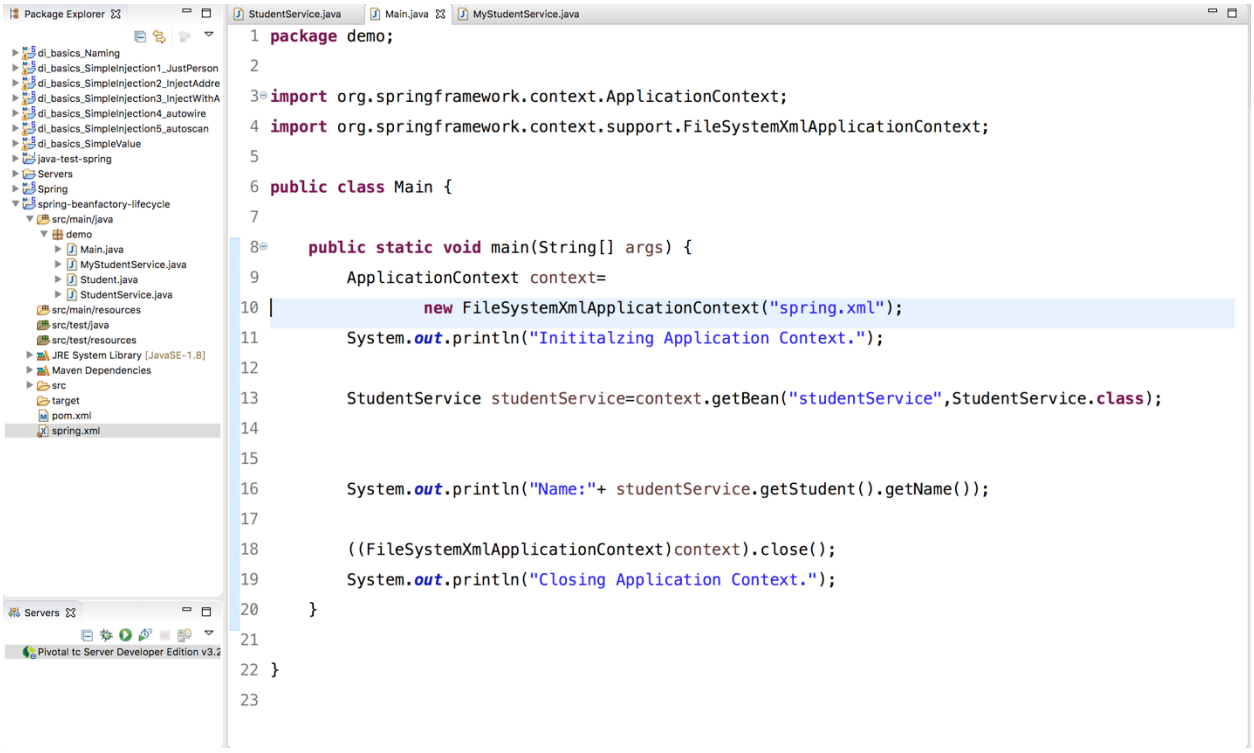
နောက်ဆုံးတွင် Custom destroy method ကိုခေါ်ဆိုခြင်းဖြင့် BeanFactory ၏ Callback Methods များခေါ်ဆိုခြင်းနိဂုံးသို့ရောက်သည်။

```
32
33
34     public void customInit() {
35         System.out.println("StudentService customInit");
36     }
37
38
39     public void customDestory() {
40         System.out.println("StudentService customDestory");
41     }
42
43
44 }
45
```




BeanFactory , ApplicationContext စသည့် Spring Bean Container များကိုပြန်လည်ခေါ်ဆိုအသုံးပြုပုံများကိုလေ့လာကြပါစို့။

1. BeanConfigurationFile (or) XmlBeanFactory သည် Application level တွင်ရှိနေလျှင် အောက်ပါအတိုင်းခေါ်ဆိုရပါမည်။ဥပမာ။ ။




```java
package demo;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.FileSystemXmlApplicationContext;

public class Main {

    public static void main(String[] args) {
        ApplicationContext context=
                new FileSystemXmlApplicationContext("spring.xml");
        System.out.println("Inititalzing Application Context.");

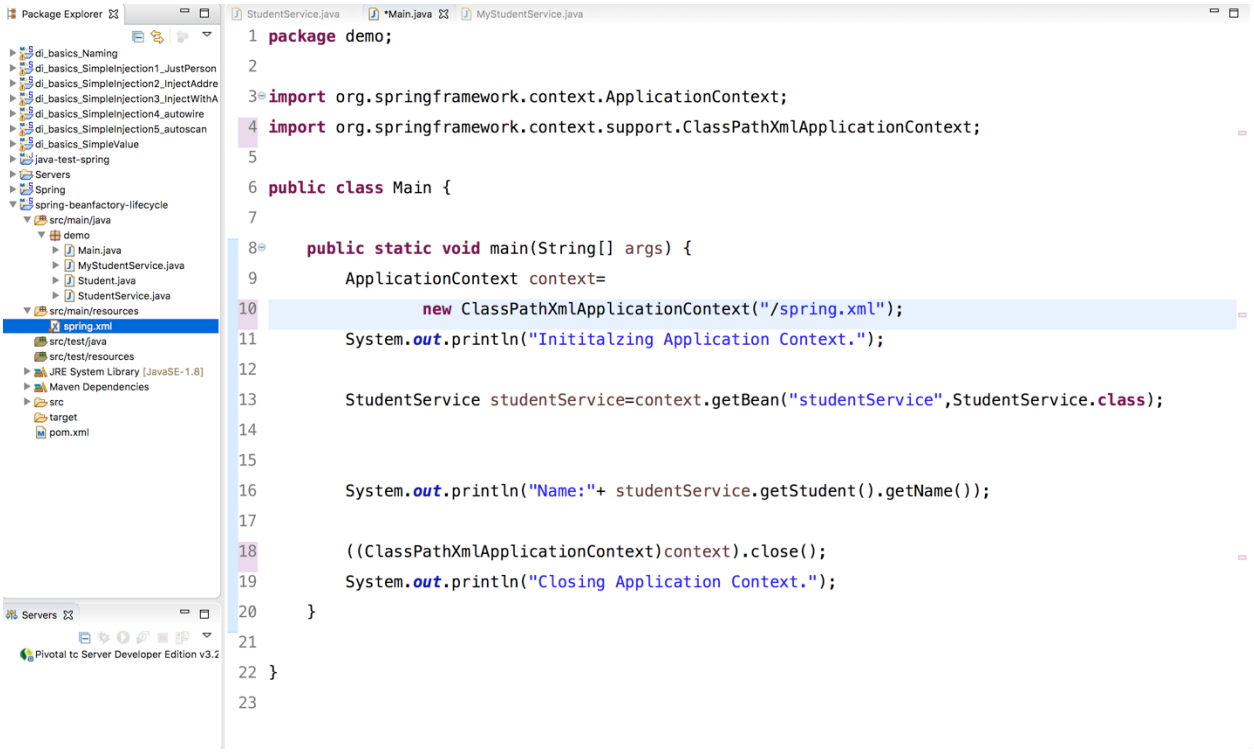
        StudentService studentService=context.getBean("studentService",StudentService.class);


        System.out.println("Name:"+ studentService.getStudent().getName());

        ((FileSystemXmlApplicationContext)context).close();
        System.out.println("Closing Application Context.");
    }

}
```

Spring.xml File သည် class path အတွင်းတွင်တည်ရှိလျှင် အောက်ပါအတိုင်းခေါ်ဆိုရမည်ဖြစ်သည်။




```java
package demo;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;

public class Main {

    public static void main(String[] args) {
        ApplicationContext context=
                new ClassPathXmlApplicationContext("/spring.xml");
        System.out.println("Inititalzing Application Context.");

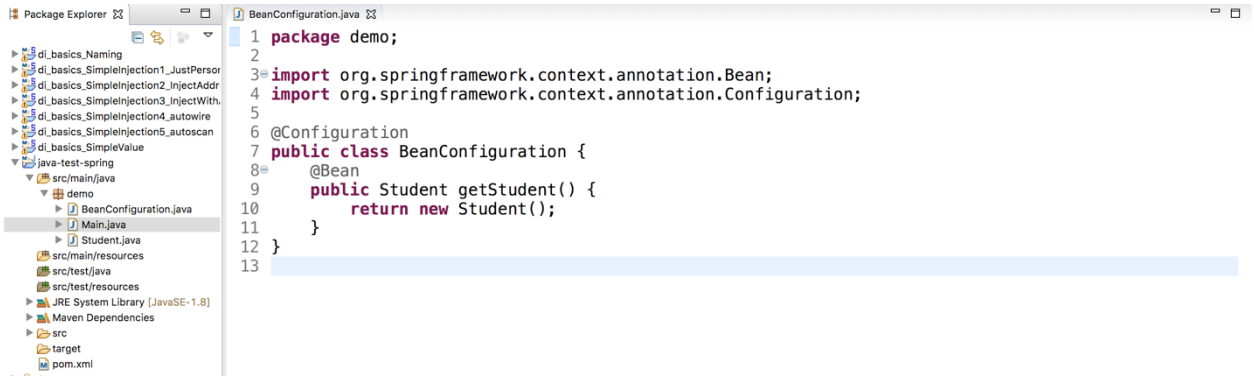
        StudentService studentService=context.getBean("studentService",StudentService.class);

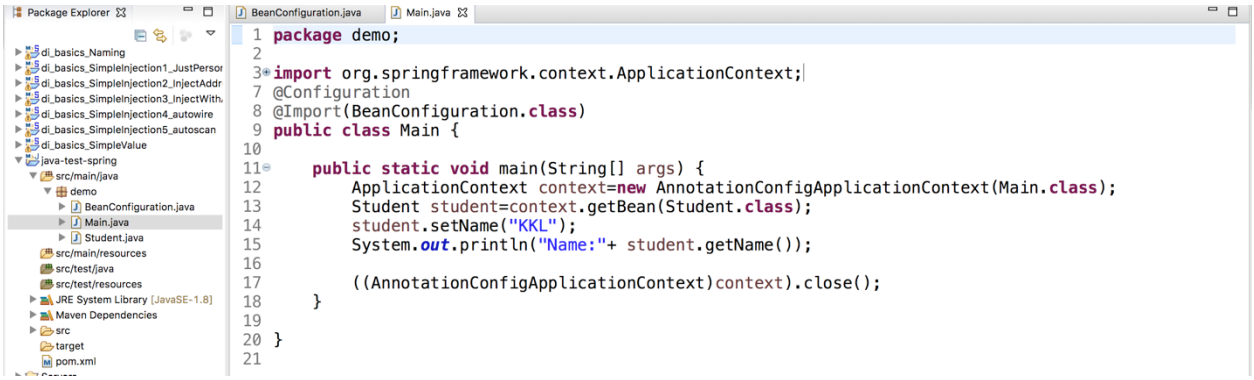

        System.out.println("Name:"+ studentService.getStudent().getName());

        ((ClassPathXmlApplicationContext)context).close();
        System.out.println("Closing Application Context.");
    }

}
```

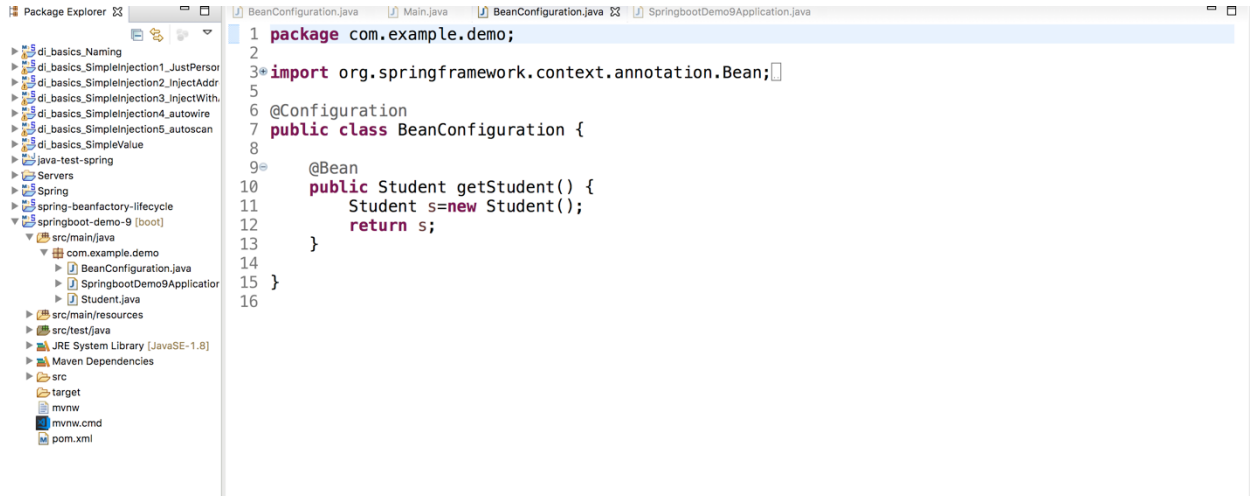
Spring Bean Configruation File ကိုထည့်သွင်းရေးသားခြင်းမပြုဘဲ Java class များဖြင့်လည်းရေးသားနိုင်သည်။ ထိုအခါတွင် အောက်ပါအတိုင်းခေါ်ဆိုနိုင်ပါသည်။

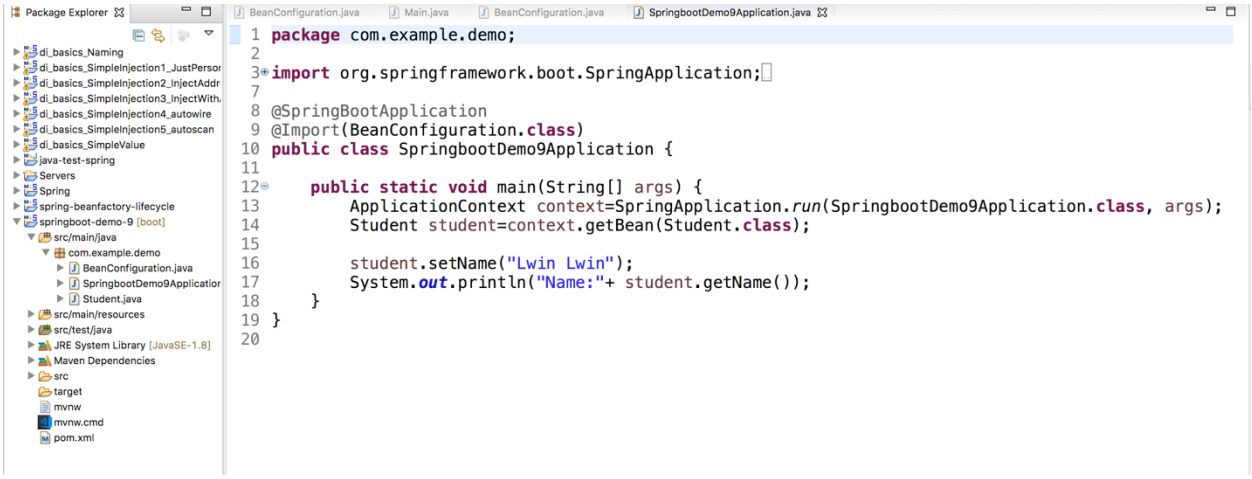



2012 October တွင် Mike Youngstrom သည် Spring Framework အတွက် containerless web application architecture ကိုဖန်တီးသည်။ Spring Container ကို အစပြုပြီး Web Container Services များကို main method မှပင်အလုပ်လုပ်ဆောင်နိုင်ကြောင်း ပြောဆိုသည်။ထိုအကြံပြုချက်မှပင် 2014 April တွင် Spring boot 1.0.0 ကိုဖြန့်ချိနိုင်သည်။

Spring Boot တွင် ApplicationContext ,BeanFactory တို့ကို အောက်ပါအတိုင်းခေါ်ဆိုနိုင်သည်။




ဤတွင် Spring Container , BeanFactory, ApplicationContext ,Spring Bean အကြောင်း နိဂုံးတံလေပြီ။